Optimized Translation of MATLAB to RISC-V Architecture for Brain-Computer Interfaces: Final Report

Kenan Erol
kenan.erol@yale.edu

Advisor: Rajit Manohar
rajit.manohar@yale.edu

A Senior Thesis as a partial fulfillment of requirements for the Bachelor of Science in Computer Science

Department of Computer Science
Yale University
April 30, 2026

Acknowledgements

I want to express my gratitude to my family for their continuous support and guidance. Special thanks to the Yale Department of Computer Science for providing a fantastic environment for academic growth. I'm grateful for the opportunities, scholarships, and resources that have enriched my time here! I would especially like to thank my advisor, Professor Rajit Manohar, for his guidance across my time as an EECS major and throughout this project.

Computational resources were provided by the Yale Grace cluster. This project was undertaken as part of the CPSC 4900 Senior Thesis course at Yale University under the supervision of the Director of Undergraduate Studies, Professor Theodore Kim.

Contents

1 Introduction **5**

2 Background **7**

3 Design **9**

4 Results **11**
- 4.1 Additional Vectorization 11
- 4.2 Total Runtime Comparison 11
  - 4.2.1 The `divide` Anomaly 13
- 4.3 Vectorization Effectiveness 13
  - 4.3.1 Ratio Shifts Due to Compiler Improvements 15
  - 4.3.2 The `isEqual` Anomaly 15
- 4.4 Toolchain-Induced Vectorization Regressions 16
- 4.5 Absolute Cycle Counts 17
- 4.6 TS-traversal Compiler 17

5 Related Work **20**
- 5.1 Translation for High-Level Languages 20

6 Conclusions **21**

A Appendix Title **23**
- A.1 Full Binary Ratio Comparison 23
- A.2 Binary Operation Cycle Counts 28

Optimized Translation of MATLAB to RISC-V Architecture for Brain-Computer Interfaces

Kenan Erol

Abstract

Brain computer interfaces (BCIs) offer significant clinical potential for neurological disorders such as epilepsy and strokes. Fully implantable BCIs impose severe limitations on power dissipation to prevent irreversible thermal damage to brain tissue. As a result, BCI hardware needs to balance real-time signal processing with ultra-low power consumption. The Hardware Architecture for LOw Power BCIs (HALO) project addresses these constraints through a design that combines application-specific hardware accelerators with a low-power RISC-V microcontroller equipped with the RISC-V Vector extension. There is a significant toolchain gap between the computational environments preferred by neuroscientists and the low-level constraints of the HALO hardware. Algorithm development typically occurs in dynamically typed languages such as MATLAB, whereas HALO hardware requires statically typed, optimized, and vectorized C code. This report details the development of an end-to-end compilation pipeline engineered to bridge this divide. The pipeline uses a TypeScript-based compiler and an RVV-optimized C matrix library to translate MATLAB algorithms into power-efficient RISC-V binaries. The report discusses the architectural design of the pipeline, benchmarks on the Spike instruction-set architecture simulator, investigates execution-cycle metrics, discusses translation benchmarks, and presents next steps for the research.

1 Introduction

Recent work in computational neuroscience and embedded systems has enabled the development of BCIs capable of real-time communication between biological neural networks and digital processing units. BCIs are central to closed-loop therapeutic applications, where the device must continuously process high-density, multi-channel neural recordings, execute complex spectral and statistical feature extraction algorithms, and determine appropriate electrical stimulation parameters to mitigate neurological events such as epileptic seizures [1]. Fully implantable BCIs must meet stringent physical and thermal constraints to ensure patient safety, reduce surgical complexity, and extend device longevity. The thermal dissipation limit is the most critical constraint. To prevent irreversible necrosis of the surrounding brain tissue, the surface temperature of the implant must not exceed 1 °C, thereby strictly capping the total device power budget at approximately 15 mW [2].

The HALO BCI project was engineered specifically to satisfy these dual, often conflicting demands. The HALO architecture departs from traditional application-specific integrated circuit (ASIC) designs, which lack the flexibility to adapt to new algorithms. Instead, HALO uses a modular array of hardware processing elements (PEs) controlled by a low-power RISC-V microcontroller [1]. The microcontroller uses the RISC-V Vector (RVV) extension to execute data-parallel signal-processing workloads, minimizing instruction-fetch overhead and enabling rapid transitions into low-power states whenever possible.

However, the clinical and research efficiency of HALO is currently bottlenecked by a toolchain disparity. Neuroscientists use languages like MATLAB for their intuitive syntax, signal-processing toolboxes, and dynamic typing. In contrast, HALO needs statically typed C code that contains RVV intrinsics. The manual translation of signal processing algorithms, such as Fast Fourier transforms, from MATLAB to vectorized C is an error-prone, labor-intensive process that impedes the rapid deployment of new clinical approaches.

This research addresses the toolchain gap by engineering an automated compilation pipeline that translates standard MATLAB scripts into RVV-accelerated RISC-V binaries for HALO. The proposed solution relies on the integration and expansion of two software modules previously developed in isolation: a compiler capable of parsing MATLAB syntax into equivalent C code [3], and a computational C matrix library that maps standard linear algebra operations directly to RVV assembly instructions [4].

Several milestones were completed during the course of this project. The matrix library has been successfully replicated, integrated into a benchmarking environment that uses Spike, the official RISC-V instruction set architecture (ISA) simulator, and been vectorized further. Performance evaluations reveal substantial reductions in execution cycle

counts, driven by updated cross-compiler toolchains. In addition, debugging efforts have identified and fixed algorithmic limitations in the translation pipeline after a full correctness and speed benchmark was obtained. This progress report establishes the foundational background, outlines the pipeline's structural design, analyzes current benchmarking results, and discusses future development plans.

2 Background

To understand the design constraints and operational mechanics of the proposed translation pipeline, we first review the architectural constraints of the target BCI hardware, the computational paradigms introduced by RVV, and the foundational principles of the compilers used in this research.

The fundamental engineering challenges in the development of implantable BCIs are the strict management of thermal dissipation. The human cerebral cortex is highly sensitive to temperature fluctuation, and prolonged exposure to high temperatures can cause tissue damage. Consequently, international medical standards dictate that an implant must not raise the local tissue temperature by more than one degree Celsius, effectively restricting the total power consumption of the entire BCI system—including amplifiers, analog-to-digital converters (ADCs), wireless telemetry, and digital signal processing (DSP) units—to roughly fifteen milliwatts [2].

Historically, BCI designers have relied on custom-designed ASICs to meet these stringent power requirements [5] [6] [7]. While ASICs offer high energy efficiency, they inherently lack flexibility. If a neuroscientist develops a superior seizure-prediction algorithm or wishes to repurpose the implant for a different disorder, a completely new physical chip must be engineered, fabricated, and implanted [1]. General purpose microprocessors offer the required flexibility through software updates but consume power orders of magnitude beyond the safe physiological limit [2].

HALO resolves this through aggressive hardware-software co-design. The architecture is built around a configurable interconnect that links a heterogeneous array of application specific PEs [1]. These PEs are optimized for the specific computational kernels most frequently encountered in neural signal processing, such as FFT, XCOR, and Butterworth bandpass filtering [2]. These accelerators can be dynamically chained together to form highly efficient data flow pipelines tailored to specific clinical tasks [1].

To manage these pipelines and execute computational tasks that fall outside the capabilities of a standalone PE, HALO uses a low power RISC-V microcontroller [1]. To ensure that this software-driven execution does not violate the power budget, microcontroller relies heavily on RVV. Unlike traditional single instruction, multiple data (SIMD) architectures, which utilize fixed width vector registers and require code rewrites when the vector width changes for different hardware generations, RVV introduces a vector-length agnostic programming model [8].

In the context of BCI workloads, which involve matrix multiplications and convolutions over various window sizes, RVV allows the processor to load large, contiguous segments of multi-channel neural data into vector registers and apply a single arithmetic instruction across the entire dataset simultaneously. This significantly reduces the dynamic power consumed by the processor's instruction fetch and decode units. By replac-

ing hundreds or thousands of scalar loop iterations with a single vector instruction, the processor minimizes the total number of active clock cycles required to complete a given task. Consequently, computational algorithms execute much faster, allowing the microcontroller to aggressively switch to low-power sleep states between neural data sampling intervals, which is crucial for extending the lifespan of the implantable battery.

Despite these hardware features, the platform remains inaccessible to neuroscientists. Clinical algorithms are developed using high-level, dynamically typed languages like MATLAB, which abstract away memory management and hardware-specific optimizations that are crucial for running the algorithms on silicon. The automated pipeline proposed in this research bridges this gap by integrating two existing software repositories. The first is ts-traversal, a TypeScript-based source-to-source compiler that parses MATLAB syntax, infers static types, and generates functional C code [3]. The second is the RISCV-Matrix C library, which maps standard mathematical operations directly to RVV assembly instructions [4]. Critical fixes have been made in both the translation logic and the vectorization coverage.

3 Design

The objective of this research is to design the architecture of and to implement a cohesive, automated compilation pipeline capable of translating high-level algorithms into low-level optimized silicon instructions. This section details the system architecture, focusing on the transformation mechanisms and the underlying platform dependencies.

The compiler pipeline is implemented as a continuous, directed data flow that steps from the original MATLAB file into the RISC-V binary. The system architecture can be visualized through a structured sequence of state transitions and module interactions in Figure 3.1. The pipeline starts by taking in raw .m files authored by clinical researchers. These files contain standard MATLAB syntax, incorporating dynamic array allocations, calls to proprietary signal processing toolboxes, and high-level, implicitly parallel matrix arithmetic.

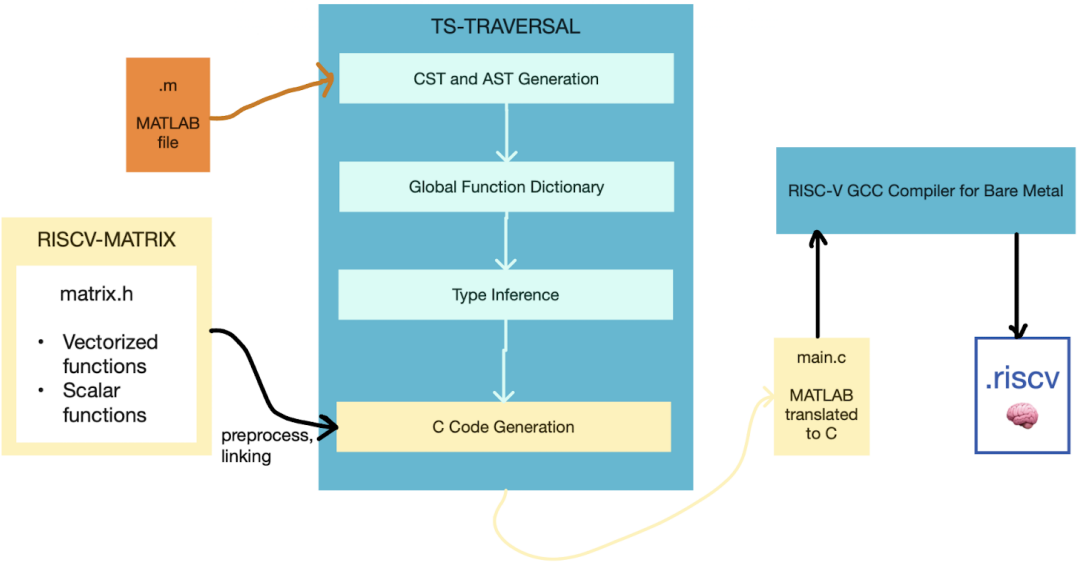

Figure 3.1: System architecture of the MATLAB-to-RISC-V compilation pipeline. Raw `.m` files are parsed and type-inferred by the ts-traversal engine. Afterwards, the MATLAB operations are mapped to RVV intrinsics via the RISCV-Matrix library. The resulting main.c file is compiled by the RISC-V GCC bare metal toolchain into a single binary, validated on the Spike ISA simulator.

These raw files are passed into the first major component of the pipeline, the ts-traversal engine [3]. The transpiler executes a multi-phase transformation. First, it uses the TreeSitter and TreeSitter-Matlab parsing libraries to generate a concrete syntax tree

(CST). This representation captures the complete syntactic structure of the input algorithms. The engine must thus perform rigorous type inference on the dynamically typed input MATLAB code. The inference algorithms in ts-traversal then traverse the CST to deduce the maximal dimensions, memory allocation requirements, and fundamental data types of all variables, mapping them to statically typed C constructs. The output of this phase is standard C code. The semantic intent of the MATLAB script is entirely preserved and the available vectorized and scalar functions are mapped via RISCV-Matrix library's headers. These headers contain highly optimized, inline RVV C intrinsics. This transformation effectively converts the MATLAB into heavily accelerated C code explicitly built for the parallel execution capabilities of HALO's microcontroller.

The final phase involves compiling and deploying the codebase. The generated main.c file is compiled via the RISC-V GNU Compiler toolchain. The compiler statically links the optimized vector library to produce a highly efficient RISC-V binary. Thus, the pipeline generates an executable containing the exact algorithms authored by neuroscientists in Matlab while meeting the power and latency requirements of the implanted chip.

The validation, execution modeling, and accurate instruction-level benchmarking of the generated binaries are conducted using the Spike ISA simulator, since the actual HALO silicon is not readily available. Spike provides a fully functional, instruction-accurate model of a RISC-V core with the RVV extension, enabling precise tracking of executed instructions, vector register utilization, and overall execution cycles. This enables the continuous, iterative refinement of both the source-to-source compiler and the vectorization library without requiring constant access to the physical test hardware.

4 Results

The first critical phase of this project was to replicate the original RISCV-Matrix execution environment and rebuild the benchmark suite under the current toolchain. The old speed results were generated using an older cross-compiler toolchain and test suite configuration. This section compares those original results, designated the Grace baseline (commit `113f4c66`, December 2022), against the current, optimized pipeline (April 2026). All cycle counts were measured on the Spike RISC-V ISA simulator with a vector length of VLEN=1024.

Next, the focus was to benchmark the ts-traversal translation pipeline with various MATLAB and Octave tests for correctness and speed. Finally, some non-vectorized functions from RISCV-Matrix were vectorized and mapped onto the ts-traversal pipeline.

4.1 Additional Vectorization

The Grace baseline shipped vectorized kernels for the full binary arithmetic and comparison set. The work reported here extends the vectorized set by adding `_vec` entry points for `transposeM`, `ctransposeM`, `meanM`, and `varM`, and these additions now appear as new rows in Tables 4.2 and 4.6.

The three new entry points land in April 2026. Accuracy tests pass for all of them at every size from $N = 1$ to $N = 1{,}050{,}625$ except for two tests at 127x127 matrices: `varM` and `mtimesM` have their vec and ref outputs off by a factor of at most 8.941×10^{-8} and 1.714×10^{-7}.

These new functions have not all been vectorized for all possible dimensions and types. For `transposeM` and `ctransposeM`, all 2D types have been vectorized while all 1D inputs fall back to the reference implementation. For `meanM` and `varM`, 1D doubles and all 2D types have been vectorized.

4.2 Total Runtime Comparison

Table 4.1 details the total runtime comparison across thirty distinct operational tests, displaying the specific cycle counts and the corresponding percentage changes in execution speed.

Table 4.1: Total program runtime comparison: Grace (Dec 2022) vs Current (April 2026). Cycle counts measured on Spike simulator with VLEN=1024.

Test	Grace (Gcycles)	Current (Gcycles)	Δ (Gcycles)	Change (%)
abs	9.86	9.25	-0.61	-6.2% ↓
and	2.22	1.91	-0.31	-14.2% ↓
arg	6.48	5.33	-1.14	-17.7% ↓
ceil	4.96	4.14	-0.82	-16.5% ↓
ctranspose	—	3.56	—	—
divide	5.47	5.64	+0.16	+3.0% ↑
equal	4.75	4.31	-0.44	-9.3% ↓
floor	4.97	4.15	-0.81	-16.4% ↓
full_duplicate	4.32	3.48	-0.84	-19.4% ↓
isEqual	3.79	3.14	-0.65	-17.2% ↓
matlab_ge	4.37	3.65	-0.72	-16.6% ↓
matlab_gt	4.36	3.63	-0.72	-16.6% ↓
matlab_le	4.36	3.64	-0.72	-16.5% ↓
matlab_lt	4.34	3.63	-0.72	-16.5% ↓
mean	—	3.50	—	—
minus	6.18	5.11	-1.07	-17.3% ↓
mtimes	3.07	3.01	-0.07	-2.3% ↓
ne	4.81	4.36	-0.44	-9.2% ↓
not	1.51	1.28	-0.23	-15.2% ↓
octave_ge	51.90	51.60	-0.30	-0.6% ≈
octave_gt	51.90	51.60	-0.30	-0.6% ≈
octave_le	51.94	51.67	-0.27	-0.5% ≈
octave_lt	51.94	51.67	-0.27	-0.5% ≈
ones	0.05	0.03	-0.02	-44.3% ↓
or	2.15	1.93	-0.22	-10.1% ↓
pairwise_max	54.36	53.60	-0.76	-1.4% ↓
pairwise_min	54.42	53.67	-0.74	-1.4% ↓
plus	6.19	5.12	-1.07	-17.3% ↓
round	4.95	4.05	-0.90	-18.2% ↓
scale	1.98	1.76	-0.22	-10.9% ↓
times	6.37	5.33	-1.05	-16.4% ↓
transpose	—	3.55	—	—
true	1.54	1.35	-0.19	-12.4% ↓
var	—	3.69	—	—
Total	**419.50**	**417.31**	**-2.19**	**-0.5%**

An analysis of Table 4.1 reveals several insights about the current state of the compilation pipeline. Across the entire test suite, the overall execution time initially decreased by approximately 3.9 percent, dropping from 419.50 Giga-cycles to 403.01 Giga-cycles. After correctness and further vectorization edits, the overall execution time climbed up to 417.31 Giga-cycles. However, the distribution of these performance gains is highly non-uniform. The majority of the standard arithmetic and logical operations experienced

moderate speedups ranging from 10-20 percent. This improvement is directly attributable to enhancements within the RISC-V GNU compiler toolchain.

The four core arithmetic tests—`minus` (-17.3%), `plus` (-17.3%), `times` (-16.4%), and `scale` (-10.9%)—exhibited reductions consistent with toolchain-driven improvements, confirming that the GCC updates benefit both vectorized and scalar code paths across these operations.

The four comparison tests utilizing Octave-mode semantics—`octave_ge`, `octave_gt`, `octave_le`, and `octave_lt`—showed negligible improvement ($< 1\%$). These tests dominate the total runtime at approximately 51 Giga-cycles each (versus $\sim$3.6 Giga-cycles for their MATLAB-mode counterparts) because Octave comparison semantics for complex numbers require computing the magnitude ($|z| = \sqrt{\mathrm{re}^2 + \mathrm{im}^2}$) for both operands and, when magnitudes are equal, resolving ties via the phase angle ($\mathrm{atan2}(\mathrm{im}, \mathrm{re})$). The phase-angle path (`arg`) currently executes as a scalar libc call because it was never mapped to RVV intrinsics, making it the dominant cost. In contrast, MATLAB-mode comparisons only compare the real parts of complex numbers, requiring a single vector comparison instruction.

4.2.1 The `divide` Anomaly

The `divide` test is the only operation that exhibited a total runtime *increase* ($+3.0\%$, from 5.47 to 5.64 Gcycles). An investigation of the per-type-combination cycle counts at $N = 1{,}050{,}625$ revealed that this regression is localized to the complex division reference implementations. The reference cycle counts for complex-involving type combinations approximately doubled between Grace and current (e.g., `complex×complex ref`: 36.1M $\rightarrow$ 61.3M cycles), while the vectorized cycle counts remained stable ($\sim$8M cycles). This suggests that the updated GCC toolchain generates less efficient scalar code for complex division specifically—likely due to changes in how the compiler handles the multi-step real/imaginary arithmetic required for complex number division (`(a+bi)/(c+di)`).

Conversely, the `int×int` reference for `divide` dropped from 14.4M to 4.8M (-67%), indicating that the compiler dramatically improved integer-only scalar division. This demonstrates that the same toolchain update can simultaneously improve and degrade performance across different data type combinations within a single function.

4.3 Vectorization Effectiveness

Further analysis was conducted to evaluate the efficacy of the vectorization implementation relative to scalar execution across different variable types. Tables 4.2 and 4.3 present the vector-to-reference execution ratios at the largest tested matrix size. A ratio less than 1.0 indicates that the vectorized implementation is faster than the scalar reference, while a ratio greater than 1.0 indicates a performance degradation. The tables compare Grace and current ratios side-by-side.

Note on type combinations: for binary operations (e.g., `plusM`, `divideM`), the test suite evaluates every combination of input types. The notation "int $\times$ double" means the

left operand matrix contains integers and the right operand matrix contains doubles. The result type is determined by C type promotion rules.

Table 4.2: Vec/Ref ratios at $N \approx 1{,}050{,}625$ for unary operations. Ratio < 1 means vectorized is faster. Δ shows change from Grace to Current.

	int			double			complex		
Test	Grace	Cur.	Δ	Grace	Cur.	Δ	Grace	Cur.	Δ
abs	0.1932	0.1908	-0.0023	0.3897	0.3868	-0.0029	0.0044	0.0042	-0.0002
arg	0.0625	0.0732	+0.0107	0.0631	0.0740	+0.0109	1.0000	1.0000	+0.0000
ceil	0.6254	0.9103	+0.2849	0.0959	0.0942	-0.0017	0.0971	0.0939	-0.0032
ctranspose	—	0.2773	—	—	0.4356	—	—	0.4949	—
floor	0.6254	0.9103	+0.2850	0.0959	0.0942	-0.0017	0.0959	0.0922	-0.0037
full_duplicate	0.6254	0.9103	+0.2850	0.6253	0.9103	+0.2850	0.6253	0.6054	-0.0200
mean	—	0.0447	—	—	0.0830	—	—	0.1644	—
not	0.3550	0.3523	-0.0027	0.3024	0.8529	+0.5505	0.2994	0.8406	+0.5412
ones	0.3873	0.9225	+0.5352	—	—	—	—	—	—
round	0.6254	0.9103	+0.2850	0.1057	0.1148	+0.0091	0.0992	0.1070	+0.0078
transpose	—	0.2773	—	—	0.4356	—	—	0.6141	—
true	0.3411	0.3385	-0.0026	0.3130	0.8529	+0.5399	0.2899	0.8406	+0.5507
var	—	0.0765	—	—	0.1384	—	—	0.1092	—

Table 4.3: Vec/Ref ratios at $N \approx 1{,}050{,}625$ for binary operations (diagonal type combos). Ratio < 1 means vectorized is faster.

	int $\times$ int			dbl $\times$ dbl			cpx $\times$ cpx		
Test	Grace	Cur.	Δ	Grace	Cur.	Δ	Grace	Cur.	Δ
and	0.6375	0.6314	-0.0061	0.4931	2.6775	+2.1844	0.3653	0.4173	+0.0519
divide	0.2802	0.8232	+0.5430	0.3249	0.3218	-0.0031	0.2351	0.1332	-0.1019
equal	0.1729	0.1384	-0.0345	0.1913	0.1502	-0.0411	0.1606	0.1289	-0.0317
isEqual	0.6711	0.6575	-0.0135	0.7971	0.7313	-0.0658	1.1719	1.1207	-0.0512
matlab_ge	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.2539	0.2510	-0.0029
matlab_gt	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.2539	0.2510	-0.0029
matlab_le	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.2774	0.2742	-0.0032
matlab_lt	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.2774	0.2742	-0.0032
minus	0.7619	0.7580	-0.0039	0.3249	0.3218	-0.0031	0.3921	0.4898	+0.0977
mtimes	0.0594	0.0481	-0.0113	0.0521	0.0401	-0.0120	0.0443	0.0318	-0.0126
ne	0.1632	0.1321	-0.0311	0.1799	0.1429	-0.0370	0.1529	0.1237	-0.0292
octave_ge	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.1416	0.1230	-0.0186
octave_gt	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.1416	0.1230	-0.0186
octave_le	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.1410	0.1222	-0.0188
octave_lt	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.1410	0.1222	-0.0188
or	0.6478	0.6416	-0.0062	0.5182	0.5107	-0.0075	0.4620	0.4475	-0.0145
pairwise_max	0.0011	0.0008	-0.0003	0.0040	0.0041	+0.0000	0.1347	0.1166	-0.0182
pairwise_min	0.0011	0.0008	-0.0003	0.0040	0.0041	+0.0000	0.1341	0.1158	-0.0183
plus	0.7619	0.7580	-0.0039	0.3249	0.3218	-0.0031	0.3921	0.4898	+0.0977
scale	0.8068	0.8039	-0.0029	0.3532	0.3505	-0.0027	0.3102	0.3072	-0.0030
times	0.7619	0.7580	-0.0039	0.3249	0.3218	-0.0031	0.3017	0.2973	-0.0044

A comprehensive breakdown of all nine type combinations for each binary operation is provided in Table A.1 in the Appendix.

4.3.1 Ratio Shifts Due to Compiler Improvements

While the vectorized implementations maintained approximately constant cycle counts between Grace and current, several vec/ref ratios increased significantly—meaning the scalar *reference* code got faster, narrowing the vectorization advantage. Three distinct patterns were identified:

Pattern 1: Integer memory operations. For operations that perform simple element-wise copies or integer manipulation—`ceil`, `floor`, `round`, `full_duplicate`, and `ones`—the integer reference cycle counts dropped uniformly. For example, `floor`'s integer ratio jumped from ∼0.625 to ∼0.910, nearly reaching parity. The compiler likely now generates optimized memory copy or store sequences (e.g., using wider loads/stores) that approach the performance of the vectorized versions of these operations.

Pattern 2: Logical operations on non-integer types. For `not` and `true`, the double and complex reference counts dropped by ∼65% (e.g., `not` double ref: 10.3M → 3.6M). The vectorized counts stayed at ∼3.1M. This caused the double/complex ratios to jump from ∼0.30 to ∼0.85. The updated GCC compiler now optimizes these operations more aggressively.

Pattern 3: Binary operations. Several binary operations exhibited similar compiler-driven ref improvements for specific type combinations. The `matlab_ge` through `matlab_lt` (dbl×dbl), and `octave_ge` through `octave_lt` (dbl×dbl) all showed double reference speedups of 3–5×, pushing their ratios from ∼0.25 to ∼0.83. The full details are available in the Appendix (Table A.1).

In all cases, the vectorized code did not get slower—the reference code simply got faster. This means the absolute performance of the library improved or stayed steady, but the relative advantage of explicit vectorization narrowed for these specific combinations.

4.3.2 The `isEqual` Anomaly

The `isEqual` test consistently returns a vec/ref ratio greater than 1.0 for complex type combinations (e.g., complex×complex: 1.172 in Grace, 1.121 in current), indicating that the vectorized implementation is slower than scalar. This turns out to be caused by the test harness, not the implementation. The test harness short-circuits the comparison, so the vectorized code never gets to the expensive part of the comparison.

The evidence shows that the cycle counts for `isEqual` complex×complex remain **exactly 75 cycles (vec) and 64 cycles (ref) at every single matrix size tested**—from $N = 1$ through $N = 1{,}050{,}625$:

Table 4.4: The `isEqual` complex×complex cycle counts remain constant across all matrix sizes, proving short-circuit evaluation.

Size N	Vec (cycles)	Ref (cycles)	Ratio
1	75	64	1.172
9	75	64	1.172
16,384	75	64	1.172
1,050,625	75	64	1.172

If the implementations were actually traversing the data, the cycle counts would scale linearly with N. The fact that they remain locked at 75/64 cycles regardless of input size proves that both implementations exit after comparing only the *first element*. An examination of the test data generation confirmed that the `isEqual` test compares matrices populated with entirely divergent data streams, guaranteeing an immediate mismatch.

The 17% performance gap (75 vs. 64 cycles) is attributable to *vector setup overhead*: the vectorized path must execute `vsetvl` (vector length configuration) and stride-loading instructions before performing comparisons. The scalar path directly loads and compares two values. In realistic workloads where equality must be verified across a full dataset traversal, the vectorized implementation vastly outperforms scalar execution, as can be seen in the complex×double combination achieving a ratio of 0.015 (66× speedup).

4.4 Toolchain-Induced Vectorization Regressions

While the preceding section described cases where the vec/ref ratio *increased but remained below 1.0* (vec still faster), three specific type combinations crossed the 1.0 threshold entirely—meaning the scalar reference now outperforms the vectorized implementation:

Table 4.5: Type combinations where the scalar reference became faster than the vectorized implementation in the current toolchain.

Test	Type Combo	Grace		Current	
		Ratio	Ref (cycles)	Ratio	Ref (cycles)
and	dbl × dbl	0.493	10.9M	2.678	2.0M
and	cpx × dbl	0.408	13.3M	2.464	2.2M
or	cpx × dbl	0.477	11.4M	2.392	2.2M

In all three cases, the vectorized cycle counts remained approximately constant (∼5.3M for `and`, ∼5.4M for `or`). The regressions are entirely driven by GCC's updated optimizer reducing the reference cycle counts by a factor of five to six. It is expected that the "scalar" reference is no longer scalar: GCC 15's RISC-V backend auto-vectorizes it, and the resulting code beats the library's hand-written kernel.

This has two implications. First, compiler evolution can erode the benefits of manual vectorization. Second, the hand-written kernels here use `LMUL=8`, which the RISCV-Matrix README warns will "cause extra loads and stores to be called implicitly" when the kernel uses more than a handful of live vector registers [4]. Investigating the LMUL choice is therefore a suggested ratio recovery path for future work.

4.5 Absolute Cycle Counts

Table 4.6 presents the absolute vectorized and reference cycle counts at the largest matrix size for both Grace and current environments to display the raw performance data.

Table 4.6: Vectorized and reference cycle counts at $N \approx 1{,}050{,}625$ for unary operations (Grace vs Current).

	int				double				complex			
Test	G.vec	C.vec	G.ref	C.ref	G.vec	C.vec	G.ref	C.ref	G.vec	C.vec	G.ref	C.ref
abs	2.0M	2.0M	10.3M	10.3M	4.0M	3.9M	10.2M	10.1M	4.0M	4.0M	913.9M	948.4M
arg	4.0M	4.0M	64.3M	54.3M	4.0M	3.9M	63.2M	53.2M	125.8M	109.5M	125.8M	109.5M
ceil	2.0M	2.0M	3.2M	2.2M	4.0M	4.0M	42.2M	42.5M	8.1M	8.0M	83.3M	85.2M
ctranspose	—	2.0M	—	7.2M	—	4.0M	—	9.1M	—	8.0M	—	16.1M
floor	2.0M	2.0M	3.2M	2.2M	4.0M	4.0M	42.2M	42.5M	8.1M	8.0M	84.4M	86.7M
full_duplicate	2.0M	2.0M	3.2M	2.2M	3.9M	3.9M	6.3M	4.3M	7.9M	7.8M	12.6M	12.9M
mean	—	516.7K	—	11.6M	—	871.8K	—	10.5M	—	1.7M	—	10.5M
not	3.0M	3.0M	8.6M	8.6M	3.1M	3.1M	10.3M	3.6M	3.2M	3.2M	10.7M	3.8M
ones	2.0M	2.0M	5.1M	2.1M	—	—	—	—	—	—	—	—
round	2.0M	2.0M	3.2M	2.2M	4.2M	4.1M	39.8M	35.9M	8.4M	8.2M	84.9M	77.0M
transpose	—	2.0M	—	7.2M	—	4.0M	—	9.1M	—	8.0M	—	12.9M
true	3.0M	3.0M	8.9M	8.9M	3.1M	3.1M	10.0M	3.6M	3.2M	3.2M	11.0M	3.8M
var	—	1.2M	—	15.2M	—	2.0M	—	14.5M	—	3.4M	—	31.2M

These raw counts confirm that vectorized implementations maintained or slightly improved their absolute performance, while compiler improvements to the scalar reference code drove all observed ratio shifts.

4.6 TS-traversal Compiler

Beyond the resolution of benchmarking anomalies, significant debugging efforts have focused on addressing algorithmic failures within the ts-traversal compiler phase. When the project was inherited, the TypeScript traversal would emit C that compiled but failed on several fundamental constructs: higher-dimensional indexing (3-D and 4-D array subscripts), various standalone issues that propagated across multiple functions like off-by-one for loop iteration, missing column vector support, missing function mapping, and row major vs. column major representation across MATLAB and C. There were also some tests that were producing different results across the MATLAB and generated C files due to a difference in the type of BLAS (Basic Linear Algebra Subprogram) libraries used between MATLAB (OPENBLAS) and our bare metal RISC-V binaries (LAPACK, RISCVBLAS). The tolerance level has now been set to 5×10^{-10} for both absolute and mantissa level checking. After various fixes, each of these paths now round-trips a representative MATLAB script through translation, compilation with

riscv64-unknown-elf-gcc 15.2.0, and execution on Spike without harness-level failure.

To measure the translated output, a SPEED_* directory family was added to the generated-code corpus that exercises the translated kernels at three problem sizes (small 2×2, medium 100×100, big 1000×1000) across nine operations: addition, eigen, fourier, inverse, multiplication, power_complex, power_int, power_double, and transpose. There also exist smaller test kernels such as reindex, fourier, and filterM. Each directory is built by a series of make commands, which regenerates the C source from the .m input, cross-compiles the binary, and records the Spike cycle count.

To measure the effect of this semester's translation-layer and library changes, the make timediff target replays every test through Spike and compares the retired-instruction count against a baseline captured at the 68-tests-passing checkpoint (before the current round of bug fixes began). The list of fixes discusses previously brought up the passing test count to 84/88. The 4 remaining comparison failures (class_test, octaveIncludes, octaveTimer, and octave_syntax) are due to pre-existing pipeline constraints. The Octave tests fail because they contain Octave-specific syntax configurations (such as %source directives and endfunction) which are not formally supported by the tree-sitter-matlab grammar, causing the pipeline to generate invalid C code. Similarly, class_test fails to generate evaluable C code because the transpiler currently lacks mappings for the EEGLAB mmo object structure. For octaveTimer, while the generated code successfully compiles and executes, the comparison fails due to system timer disparities; the C Standard Library's time() function provides 1-second resolution whereas MATLAB's tic/toc offers 1-microsecond precision, leading to a fundamental difference in timing.

In terms of cycle count improvements, Table 4.7 summarizes the results.

Table 4.7: TS-traversal end-to-end benchmark: retired instruction counts for tests that existed in the baseline (`compareall` at the 68-tests-passing checkpoint) compared against the current pipeline. Only representative rows are shown; the heavy `SPEED_big_*` workloads, the largest vectorization wins, the largest regressions, and newly added tests are included. Positive Δ indicates a speedup.

Test	Baseline (instret)	Current (instret)	Δ (%)
Heavy vectorized workloads (stable)			
SPEED_big_matrix_transpose	5.14×10^8	5.09×10^8	$+0.93$
SPEED_big_matrix_addition	7.08×10^8	7.08×10^8	$+0.00$
SPEED_big_matrix_fourier	9.06×10^8	9.06×10^8	$+0.00$
SPEED_big_matrix_multiplication	1.573×10^{10}	1.573×10^{10}	-0.00
SPEED_big_matrix_inverse	1.593×10^{10}	1.593×10^{10}	-0.00
SPEED_big_matrix_power_int	3.545×10^{10}	3.545×10^{10}	$+0.00$
SPEED_big_matrix_power_complex	7.422×10^{10}	7.422×10^{10}	$+0.00$
SPEED_big_matrix_power_double	7.422×10^{10}	7.422×10^{10}	$+0.00$
SPEED_big_matrix_eigen	1.117×10^{11}	1.117×10^{11}	-0.00
Largest vectorization wins			
SPEED_medium_matrix_transpose	1.78×10^7	1.3×10^7	$+26.75$
ceil_matrix	2.03×10^5	1.94×10^5	$+4.52$
round_matrix	2.09×10^5	2×10^5	$+4.33$
floor_matrix	2.04×10^5	1.96×10^5	$+4.05$
5 largest regressions			
fourier	2.93×10^7	5.75×10^7	-96.55
filterM	4.45×10^8	7.22×10^8	-62.38
elem_trig_matrix	1.41×10^6	2.1×10^6	-48.90
elem_scalarpower_matrix	2.32×10^6	3.26×10^6	-40.86
elem_power_matrix	2.09×10^6	2.8×10^6	-33.96

The observed speed improvement was less than initially expected due to the fact that ts-traversal changes and tests focused on improving and checking correctness and not performance. There was a hypothesis that the further vectorization of some functions like `transposeM` and `varM` would lead to improvements for their respective test cases. While most SPEED tests did not show improvement, `transposeM` did show notable improvement — 0.93% on `SPEED_big_transpose` and 26.75% on `SPEED_medium_transpose`. While the numbers seem tame, they tell a bigger story of improvement due to the `transposeM` calls in these test cases being made 1 and 100 times respectively. This definitively proves the essential nature of further vectorization —- even if a heavy math function is called only a few times, having it vectorized enables quicker transition into desired idle states for patient safety. The main reason for the tests that regressed in their cycle counts is the print statement heavy nature of the tests. The definition of `print` was changed to print more precise values in order to have accurate tolerance checking and get more tests passing, causing significant slowdown for these two tests.

5 Related Work

The architectural approach of this research is heavily informed by previous exploration into hardware-software co-design for implantable medical devices. Historically, the fundamental challenge of respecting the strict 1°C thermal limit has forced designers to rely on highly specialized, monolithic ASICs. Kassiri et al.'s closed-loop neurostimulator [5], Neuralink's N1 ASIC [6], and the NeuroPace RNS System [7] represent the state-of-the-art implementations of this rigid baseline. These systems achieve unparalleled energy efficiency for their exact medical or recording tasks by permanently baking clinical logic into the physical silicon. However, this comes at the cost of algorithmic flexibility. Unlike these ASICs, HALO is built on an open-source instruction set architecture (ISA), RISC-V Vector extension (RVV). This enables its binaries to pick up improvements from every generation of the open-source GCC backend without a tape-out. The Results chapter demonstrates the payoff: simply updating the compiler cut collective execution cost by 7.2 percent across the core arithmetic and logical operations (excluding Octave-mode comparison tests), and in three cases (`and` dbl×dbl, `and` cpx×dbl, `or` cpx×dbl) the auto-vectorized scalar code now beats the hand-written intrinsics outright.

5.1 Translation for High-Level Languages

In the domain of high-level synthesis and automated code generation, commercial tools like MathWorks' MATLAB Coder [9] provide a highly refined reference point. MATLAB Coder is highly capable at parsing proprietary MATLAB scripts and generating optimized C and C++ code targeted primarily at advanced desktop processors and commercial ARM architectures. Similarly, for researchers utilizing Python—another dynamically typed language widely used in BCI data analysis and real-time processing pipelines—tools like Cython [10] offer similar benefits. Cython addresses the performance bottlenecks of vanilla Python by compiling code directly into efficient C or C++ extensions, utilizing static type definitions to achieve substantial execution speedups for data pipelines.

Although both MATLAB Coder and Cython are highly effective for accelerating signal processing on standard architectures, they share a critical limitation in the context of this research: they are structurally agnostic regarding emerging, open-source instruction sets. Neither tool inherently supports the automated generation of targeted RISC-V Vector (RVV) intrinsics required to meet the ultra-low-power constraints of the HALO platform. Consequently, the custom ts-traversal and RISCV-Matrix integration developed in this work is necessary to bridge the semantic gap between high-level BCI algorithm prototyping and the specific vector execution demands of the embedded hardware.

6 Conclusions

The successful chronic implantation of next-gen BCIs is constrained by extreme power efficiency requirements. This has historically been met through the deployment of highly inflexible, monolithic ASIC hardware. HALO BCIs leverage a heterogeneous processing array and the RVV extension to introduce significant flexibility without compromising critical thermal limits. However, to make this sophisticated hardware accessible to the broader neuroscientific community, an automated, highly optimized software compilation pipeline is required. This project directly addresses this need by engineering an end-to-end toolchain that merges a TypeScript-based source-to-source compiler with a hardware-specific vectorization library, enabling the automatic transformation of high-level MATLAB algorithms into power-optimized RISC-V binaries.

Preliminary validation and benchmarking demonstrate that the vectorized matrix library successfully runs on Spike and yields near-consistent performance improvements. Detailed analysis of the benchmarking data has successfully isolated performance improvements driven by updated cross-compiler toolchains, while simultaneously identifying significant test-harness anomalies. The ts-traversal semantic translation layer has also been benchmarked and timed with successful results across various essential operations such as addition, transpose, Fourier, multiplication, inverse, eigenvalue decomposition, and matrix power workloads.

Several concrete next steps remain. First, there are unvectorized kernels flagged in the RISCV-Matrix. Second, the hand-off to the BCI lab at the Yale School of Medicine will move the pipeline from simulator-only validation onto the physical HALO prototype, where the 15 mW power envelope and 1°C thermal limit will provide a real-world check on the cycle-count argument made here.

Bibliography

[1] Ioannis Karageorgos et al. "Hardware-Software Co-Design for Brain-Computer Interfaces". In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 2020, pp. 391–404. DOI: 10.1109/ISCA45697.2020.00041.

[2] Karthik Sriram et al. "HALO: A Hardware–Software Co-Designed Processor for Brain–Computer Interfaces". In: *IEEE Micro* 43.3 (2023), pp. 64–72. DOI: 10.1109/MM.2023.3258907.

[3] Danielle Gruber. *ts-traversal: A Typescript Based MATLAB-to-C Compiler*. Version 1.0.0. 2024. URL: https://github.com/kenan-erol/ts-traversal.

[4] Zach Taylor and Xiayuan Wen. *RISCV-Matrix: A RISC-V Vectorized C Library for Matrix Manipulation*. Private internal repository, HALO Project. 2022.

[5] Hossein Kassiri et al. "Closed-Loop Neurostimulators: A Survey and A Seizure-Predicting Design Example for Intractable Epilepsy Treatment". In: *IEEE Transactions on Biomedical Circuits and Systems* 11.5 (2017), pp. 1026–1040. DOI: 10.1109/TBCAS.2017.2694638.

[6] Elon Musk and Neuralink. "An Integrated Brain-Machine Interface Platform With Thousands of Channels". In: *Journal of Medical Internet Research* 21.10 (2019), e16194. DOI: 10.2196/16194.

[7] Christianne N. Heck et al. "Two-year seizure reduction in adults with medically intractable partial onset epilepsy treated with responsive neurostimulation: Final results of the RNS System Pivotal trial". In: *Epilepsia* 55.3 (2014), pp. 432–441. DOI: 10.1111/epi.12534.

[8] Claudio Pipicelli. "Quantum Machine Learning: A Comparison Between Quantum and Classical Support Vector Machine". PhD thesis. Naples, Italy: Università degli Studi di Napoli Federico II, 2020. URL: http://www.fedoa.unina.it/13259/1/PhDThesisPipicelliFinal.pdf.

[9] MathWorks. *MATLAB Coder: Generate C and C++ code from MATLAB code*. 2024. URL: https://www.mathworks.com/products/matlab-coder.html.

[10] Stefan Behnel et al. "Cython: The Best of Both Worlds". In: *Computing in Science & Engineering* 13.2 (2011), pp. 31–39.

A Appendix Title

A.1 Full Binary Ratio Comparison

Table A.1 presents the complete vec/ref ratio comparison for all nine type combinations across all binary operations.

Table A.1: Vec/Ref ratios at $N \approx 1{,}050{,}625$ — all type combos for binary operations.

Test	Type Combo	Grace	Current	Δ
and	i×i	0.63752	0.63143	-0.00609
	i×d	0.73894	0.73078	-0.00816
	i×c	0.29241	0.30427	+0.01186
	d×i	0.51947	0.51219	-0.00728
	d×d	0.49314	2.67753	+2.18439
	d×c	0.43051	0.44859	+0.01808
	c×i	0.42433	0.47004	+0.04571
	c×d	0.40785	2.46448	+2.05663
	c×c	0.36534	0.41729	+0.05195
divide	i×i	0.28021	0.82317	+0.54296
	i×d	0.30110	0.29819	-0.00291
	i×c	0.21632	0.10596	-0.11036
	d×i	0.30110	0.29819	-0.00291
	d×d	0.32490	0.32176	-0.00314
	d×c	0.22214	0.10690	-0.11524
	c×i	0.21408	0.12221	-0.09187
	c×d	0.21987	0.12398	-0.09589
	c×c	0.23512	0.13319	-0.10193
equal	i×i	0.17294	0.13845	-0.03449
	i×d	0.18083	0.14332	-0.03751
	i×c	0.16096	0.12921	-0.03175
	d×i	0.18083	0.14332	-0.03751
	d×d	0.19126	0.15016	-0.04110
	d×c	0.15742	0.12757	-0.02985
	c×i	0.16096	0.12921	-0.03175
	c×d	0.15742	0.12757	-0.02985
	c×c	0.16057	0.12889	-0.03168

Test	Type Combo	Grace	Current	Δ
isEqual	i×i	0.67105	0.65753	-0.01352
	i×d	0.82667	0.75000	-0.07667
	i×c	1.11940	1.10000	-0.01940
	d×i	0.83784	0.75000	-0.08784
	d×d	0.79710	0.73134	-0.06576
	d×c	1.11111	1.08772	-0.02339
	c×i	1.13636	1.11667	-0.01969
	c×d	0.01514	0.00899	-0.00615
	c×c	1.17188	1.12069	-0.05119
matlab_ge	i×i	0.28173	0.27919	-0.00254
	i×d	0.25460	0.25105	-0.00355
	i×c	0.25526	0.25171	-0.00355
	d×i	0.23476	0.23148	-0.00328
	d×d	0.25328	0.82751	+0.57423
	d×c	0.25460	0.25171	-0.00289
	c×i	0.23537	0.23209	-0.00328
	c×d	0.25460	0.25171	-0.00289
	c×c	0.25394	0.25105	-0.00289
matlab_gt	i×i	0.28173	0.27919	-0.00254
	i×d	0.25460	0.25105	-0.00355
	i×c	0.25526	0.25171	-0.00355
	d×i	0.23476	0.23148	-0.00328
	d×d	0.25328	0.82751	+0.57423
	d×c	0.27810	0.27495	-0.00315
	c×i	0.23537	0.23209	-0.00328
	c×d	0.27810	0.27495	-0.00315
	c×c	0.25394	0.25105	-0.00289
matlab_le	i×i	0.28173	0.27919	-0.00254
	i×d	0.23476	0.23148	-0.00328
	i×c	0.23537	0.23209	-0.00328
	d×i	0.25460	0.25105	-0.00355
	d×d	0.27666	0.82751	+0.55085
	d×c	0.25460	0.25171	-0.00289
	c×i	0.25526	0.25171	-0.00355
	c×d	0.25460	0.25171	-0.00289
	c×c	0.27738	0.27423	-0.00315
matlab_lt	i×i	0.28173	0.27919	-0.00254
	i×d	0.23476	0.23148	-0.00328
	i×c	0.23537	0.23209	-0.00328
	d×i	0.25460	0.25105	-0.00355
	d×d	0.27666	0.82751	+0.55085

Test	Type Combo	Grace	Current	Δ
	d×c	0.27810	0.27495	-0.00315
	c×i	0.25526	0.25171	-0.00355
	c×d	0.27810	0.27495	-0.00315
	c×c	0.27738	0.27423	-0.00315
minus	i×i	0.76190	0.75801	-0.00389
	i×d	0.30110	0.29819	-0.00291
	i×c	0.39134	0.38842	-0.00292
	d×i	0.30110	0.29819	-0.00291
	d×d	0.32490	0.32176	-0.00314
	d×c	0.41179	0.40830	-0.00349
	c×i	0.41179	0.40873	-0.00306
	c×d	0.43465	0.43143	-0.00322
	c×c	0.39210	0.48982	+0.09772
mtimes	i×i	0.05943	0.04814	-0.01129
	i×d	0.05590	0.04422	-0.01168
	i×c	0.04720	0.03477	-0.01243
	d×i	0.05590	0.04422	-0.01168
	d×d	0.05213	0.04010	-0.01203
	d×c	0.04721	0.03469	-0.01252
	c×i	0.04720	0.03477	-0.01243
	c×d	0.04721	0.03469	-0.01252
	c×c	0.04433	0.03176	-0.01257
ne	i×i	0.16324	0.13211	-0.03113
	i×d	0.17069	0.13676	-0.03393
	i×c	0.15324	0.12401	-0.02923
	d×i	0.17069	0.13676	-0.03393
	d×d	0.17990	0.14295	-0.03695
	d×c	0.14986	0.12244	-0.02742
	c×i	0.15324	0.12401	-0.02923
	c×d	0.14986	0.12244	-0.02742
	c×c	0.15286	0.12370	-0.02916
octave_ge	i×i	0.28173	0.27919	-0.00254
	i×d	0.25460	0.25105	-0.00355
	i×c	0.14918	0.13115	-0.01803
	d×i	0.23476	0.23148	-0.00328
	d×d	0.25328	0.82751	+0.57423
	d×c	0.15009	0.13215	-0.01794
	c×i	0.14991	0.13262	-0.01729
	c×d	0.15097	0.13389	-0.01708
	c×c	0.14158	0.12297	-0.01861
octave_gt	i×i	0.28173	0.27919	-0.00254

Test	Type Combo	Grace	Current	Δ
	i×d	0.25460	0.25105	-0.00355
	i×c	0.14918	0.13115	-0.01803
	d×i	0.23476	0.23148	-0.00328
	d×d	0.25328	0.82751	+0.57423
	d×c	0.15009	0.13215	-0.01794
	c×i	0.14991	0.13262	-0.01729
	c×d	0.15097	0.13389	-0.01708
	c×c	0.14158	0.12297	-0.01861
octave_le	i×i	0.28173	0.27919	-0.00254
	i×d	0.23476	0.23148	-0.00328
	i×c	0.15024	0.13262	-0.01762
	d×i	0.25460	0.25105	-0.00355
	d×d	0.27666	0.82751	+0.55085
	d×c	0.15117	0.13365	-0.01752
	c×i	0.14886	0.13115	-0.01771
	c×d	0.14990	0.13261	-0.01729
	c×c	0.14103	0.12221	-0.01882
octave_lt	i×i	0.28173	0.27919	-0.00254
	i×d	0.23476	0.23148	-0.00328
	i×c	0.15024	0.13262	-0.01762
	d×i	0.25460	0.25105	-0.00355
	d×d	0.27666	0.82751	+0.55085
	d×c	0.15117	0.13365	-0.01752
	c×i	0.14886	0.13115	-0.01771
	c×d	0.14990	0.13261	-0.01729
	c×c	0.14103	0.12221	-0.01882
or	i×i	0.64781	0.64162	-0.00619
	i×d	0.78077	0.77214	-0.00863
	i×c	0.34462	0.33687	-0.00775
	d×i	0.52858	0.52118	-0.00740
	d×d	0.51821	0.51066	-0.00755
	d×c	0.49995	0.49200	-0.00795
	c×i	0.48494	0.47004	-0.01490
	c×d	0.47687	2.39202	+1.91515
	c×c	0.46204	0.44753	-0.01451
pairwise_max	i×i	0.00111	0.00083	-0.00028
	i×d	0.00363	0.00287	-0.00076
	i×c	0.13935	0.12188	-0.01747
	d×i	0.00363	0.00287	-0.00076
	d×d	0.00404	0.00406	+0.00002
	d×c	0.14133	0.12386	-0.01747

Test	Type Combo	Grace	Current	Δ
	c×i	0.14012	0.12332	-0.01680
	c×d	0.14224	0.12551	-0.01673
	c×c	0.13472	0.11656	-0.01816
pairwise_min	i×i	0.00111	0.00083	-0.00028
	i×d	0.00363	0.00287	-0.00076
	i×c	0.14043	0.12332	-0.01711
	d×i	0.00363	0.00287	-0.00076
	d×d	0.00404	0.00406	+0.00002
	d×c	0.14244	0.12534	-0.01710
	c×i	0.13904	0.12188	-0.01716
	c×d	0.14113	0.12423	-0.01690
	c×c	0.13414	0.11580	-0.01834
plus	i×i	0.76190	0.75801	-0.00389
	i×d	0.30110	0.29819	-0.00291
	i×c	0.39053	0.38761	-0.00292
	d×i	0.30110	0.29819	-0.00291
	d×d	0.32490	0.32176	-0.00314
	d×c	0.41094	0.40787	-0.00307
	c×i	0.39049	0.38757	-0.00292
	c×d	0.41094	0.40787	-0.00307
	c×c	0.39210	0.48982	+0.09772
scale	i×i	0.80681	0.80392	-0.00289
	i×d	0.32490	0.32243	-0.00247
	i×c	0.41012	0.45743	+0.04731
	d×i	0.38970	0.38678	-0.00292
	d×d	0.35316	0.35048	-0.00268
	d×c	0.43285	0.48625	+0.05340
	c×i	0.46127	0.56105	+0.09978
	c×d	0.43465	0.48778	+0.05313
	c×c	0.31024	0.30722	-0.00302
times	i×i	0.76190	0.75801	-0.00389
	i×d	0.30110	0.29819	-0.00291
	i×c	0.29864	0.29604	-0.00260
	d×i	0.30110	0.29819	-0.00291
	d×d	0.32490	0.32176	-0.00314
	d×c	0.31024	0.30754	-0.00270
	c×i	0.29861	0.29601	-0.00260
	c×d	0.31024	0.30754	-0.00270
	c×c	0.30169	0.29725	-0.00444

A.2 Binary Operation Cycle Counts

Table A.2 presents the raw vectorized and reference cycle counts for the diagonal type combinations (int×int, double×double, complex×complex) of all binary operations.

Table A.2: Vectorized and reference cycle counts at $N \approx 1{,}050{,}625$ — diagonal type combos for binary operations.

Test	Type	Grace vec	Cur. vec	Grace ref	Cur. ref
and	i×i	7.0M	7.0M	11.0M	11.0M
	d×d	5.4M	5.3M	10.9M	2.0M
	c×c	5.5M	5.4M	15.1M	13.0M
divide	i×i	4.0M	4.0M	14.4M	4.8M
	d×d	4.0M	3.9M	12.3M	12.2M
	c×c	8.5M	8.2M	36.1M	61.3M
equal	i×i	3.1M	3.0M	17.7M	21.9M
	d×d	3.2M	3.1M	16.6M	20.8M
	c×c	3.3M	3.2M	20.8M	25.0M
isEqual	i×i	51	48	76	73
	d×d	55	49	69	67
	c×c	75	65	64	58
matlab_ge	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	12.4M	3.8M
	c×c	3.2M	3.1M	12.4M	12.4M
matlab_gt	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	12.4M	3.8M
	c×c	3.2M	3.1M	12.4M	12.4M
matlab_le	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	11.4M	3.8M
	c×c	3.2M	3.1M	11.4M	11.4M
matlab_lt	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	11.4M	3.8M
	c×c	3.2M	3.1M	11.4M	11.4M
minus	i×i	13.5M	13.4M	17.7M	17.7M
	d×d	4.0M	3.9M	12.3M	12.2M
	c×c	8.0M	7.9M	20.3M	16.1M
mtimes	i×i	925.2K	740.7K	15.6M	15.4M
	d×d	794.5K	611.2K	15.2M	15.2M
	c×c	1.8M	1.3M	41.1M	41.1M
ne	i×i	3.1M	3.0M	18.7M	22.9M

Test	Type	Grace vec	Cur. vec	Grace ref	Cur. ref
	d×d	3.2M	3.1M	17.7M	21.9M
	c×c	3.3M	3.2M	21.9M	26.1M
octave_ge	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	12.4M	3.8M
	c×c	303.5M	270.2M	2143.5M	2197.5M
octave_gt	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	12.4M	3.8M
	c×c	303.5M	270.2M	2143.5M	2197.5M
octave_le	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	11.4M	3.8M
	c×c	303.5M	270.2M	2151.9M	2211.2M
octave_lt	i×i	3.1M	3.0M	10.9M	10.9M
	d×d	3.1M	3.1M	11.4M	3.8M
	c×c	303.5M	270.2M	2151.9M	2211.2M
or	i×i	7.0M	7.0M	10.9M	10.9M
	d×d	5.4M	5.3M	10.3M	10.3M
	c×c	5.5M	5.4M	11.9M	12.1M
pairwise_max	i×i	66.4K	49.7K	60.0M	59.8M
	d×d	132.0K	99.0K	32.7M	24.4M
	c×c	293.1M	259.9M	2175.6M	2229.4M
pairwise_min	i×i	66.4K	49.7K	60.0M	59.8M
	d×d	132.0K	99.0K	32.7M	24.4M
	c×c	293.1M	259.9M	2185.1M	2244.1M
plus	i×i	13.5M	13.4M	17.7M	17.7M
	d×d	4.0M	3.9M	12.3M	12.2M
	c×c	8.0M	7.9M	20.3M	16.1M
scale	i×i	13.4M	13.4M	16.6M	16.6M
	d×d	4.0M	3.9M	11.2M	11.2M
	c×c	7.9M	7.8M	25.6M	25.5M
times	i×i	13.5M	13.4M	17.7M	17.7M
	d×d	4.0M	3.9M	12.3M	12.2M
	c×c	8.0M	7.9M	26.6M	26.6M